

บทที่ 2

ทฤษฎีพื้นฐาน

2.1 ความรู้เกี่ยวกับการสร้างภาพสามมิติ

1. ส่วนประกอบของ Model สามมิติ มีดังนี้

- Vertex คือ จุด (point) ซึ่งวางอยู่บนระนาบแกน โดยมีพิกัดเป็น (x, y, z)
- Vertices คือ เซตของจุดที่ใช้แทนตำแหน่งของวัตถุ เช่น Mesh ใน 3D Space
- Edge คือ เส้นที่เชื่อมต่อระหว่าง Vertex หนึ่งไปยังอีก Vertex หนึ่ง
- Polygon คือ ระนาบที่เกิดจากการนำ Edge มาวางเรียงต่อกัน โดยที่ 1 Polygon จะต้องมีย่าน้อย 3 Edge การทำภาพที่มีลักษณะโค้งจะใช้หลายๆ Polygon มาเรียงต่อกันให้ดูโค้งแทนการทำให้เป็นเส้นโค้งจริงเพื่อประหยัดเวลาในการคำนวณภาพ 3 มิติที่มีลักษณะโค้งที่มีจำนวนของ Polygon มากจะทำให้ภาพมีความโค้งมนสมจริงมากขึ้น แต่ก็ต้องใช้เวลาในการประมวลผลภาพมากยิ่งขึ้น
- Face คือ ส่วนประกอบที่อยู่ใน Mesh หรือ Poly ที่ถูกแบ่งครั้งนั่นเอง หรือก็คือ Vertices ที่มีตั้งแต่ 3 จุดขึ้นไปมาเชื่อมต่อกันเป็นรูปทรงต่างๆในแนวระนาบ Vertex เป็นตัวกำหนดมุมของ Face ทำให้ทุกๆ Vertex ใน Face จะต้องถูกกำหนดให้อยู่ในแนวระนาบ
- Mesh เกิดจากการรวมกันของ Face ที่เชื่อมต่อกัน ซึ่ง 1 Mesh สามารถมี Face ได้ตั้งแต่ 1 Face ขึ้นไป การรวมของ Face นี้ทำให้ง่ายต่อการจัดการวัตถุในการทำ Animation, Material และ Texture ชนิดการรวมกันของ Face มีดังนี้คือ

1. Fan คือ กลุ่มของรูปทรงสามเหลี่ยม ซึ่งทุกรูปมีการใช้ Vertex ร่วมกัน 1 จุด โดยกำหนดให้ Vertex นั้นๆอยู่ระหว่างกลางของสามเหลี่ยมเหล่านั้นชนิดของการรวม Face แบบ Fan นี้คล้ายกับแบบ Strip คือมีการกำหนดค่า Vertices 3 ค่าแรกสำหรับสามเหลี่ยมแรก จากนั้นในสามเหลี่ยมต่อไปก็เพียงแค่เพิ่มขึ้นมารูปละ 1 Vertex เท่านั้น

2. Strip คือ กลุ่มของสามเหลี่ยม ซึ่งแต่ละรูปจะมีการใช้เส้นร่วมกันกับสามเหลี่ยมส่วนหน้าซึ่งหมายความว่า หลังจากที่มีการกำหนดค่า Vertices 3 ค่าแรกสำหรับสามเหลี่ยมแรก จากนั้นในสามเหลี่ยมต่อไปก็เพียงแค่เพิ่มขึ้นมารูปละ 1 Vertex นั่นเอง

3. List คือ กลุ่มของสามเหลี่ยม ซึ่งทุกรูปไม่มีการใช้เส้นหรือ Vertex ร่วมกันเลย กล่าวคือ ค่า 3 ค่าของสามเหลี่ยมทุกรูปต้องกำหนดเองทั้งหมด

-Materials เป็นค่าที่กำหนดว่าจะให้ Polygon Surface เป็นอย่างไรเมื่อกระทบกับแสง โดยใน Materials มีค่าต่างๆอยู่ด้วยกัน 4 ค่า ดังนี้

1. Diffuse Color เป็นค่าของสีที่มักกำหนดเป็น RGB บอกว่า Vertices จะเป็นสีอะไรเมื่อกระทบกับแสง

2. Ambient Color เป็นสีที่ตรงข้ามกับ Diffuse Color คือเป็นสีที่จะแสดงเมื่อแสงหายไป

3. Emissive Color เป็นสีที่แสดงเป็น Effect of Fluorescence (สีส่องสว่าง) เมื่อ Verticesกระทบแสง

4. Specular Color เป็นค่าที่ทำให้วัตถุส่องสว่าง โดยต้องกำหนดค่า Power ยิ่งมากยิ่งส่องสว่าง หากค่า Power เป็น 0 วัตถุจะไม่ส่องสว่าง

2.2 Light (แสง)

แสงทำให้เกิดการเปลี่ยนแปลงของสีของ Vertices โดย Module ทำให้เกิด Vertex Normal เพราะสิ่งนี้ขึ้นอยู่กับมุมของแหล่งกำเนิดแสงตามปกติจะมีแสงสีขาว เพราะเป็นการรวมกันอย่างหนาแน่นของสีทุกสีและโดยมากมักใช้รูปแบบของ RGB ในการกำหนดสีของแหล่งกำเนิดแสงใน 3D นั้นมีการตั้งค่า RGB ของแม่สีต่างๆดังนี้

แสงสีขาว เป็น 1, 1, 1

แสงสีแดง เป็น 1, 0, 0

แสงสีน้ำเงิน เป็น 0, 0, 1

นอกจากนี้เรายังสามารถใช้ 3 สี นี้ในการผสมสีได้ ซึ่งแหล่งกำเนิดแสงมี 4 ชนิด ได้แก่

- Ambient Light คือแหล่งกำเนิดแสงที่ง่ายที่สุด เพราะไม่ต้องมีการกำหนดตำแหน่งของแหล่งกำเนิดแสง และยังให้ความสว่างทั่วทุก Object

- Point Light เป็นแหล่งกำเนิดแสงที่ทำการกระจายแสงไปทุกทิศทาง แต่ต้องระบุตำแหน่งของแหล่งกำเนิดแสง โดยไม่ต้องกำหนดทิศทางของแสง

- Directional Light เป็นแหล่งกำเนิดแสงที่มีประสิทธิภาพมากที่สุดเพราะ เป็นแหล่งกำเนิดแสงที่มีทิศทาง โดยต้องระบุตำแหน่งของแหล่งกำเนิดแสง

- Spot Light เป็นแหล่งกำเนิดแสงที่ต้องมีการระบุทั้งทิศทางและตำแหน่งของแหล่งกำเนิดแสง โดยการผลิตแสงจะเป็นรูปร่าง

ในการกำหนด Flexible Vertex Format (FVF) หากเราต้องการให้จุด vertex ของเรามี
พิกัด Normal ด้วยนั้นเราจะต้องเติม Flag D3DFVF_NORMAL ลงไปด้วย จากนั้นจึงมากำหนด
โครงสร้างข้อมูลของจุด vertex ให้มีพิกัด Normal เพิ่มเข้าไปด้วยตัวอย่างเช่น

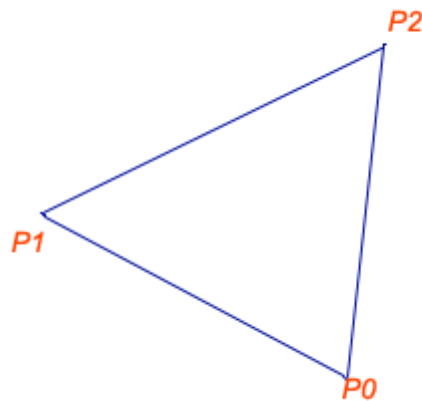
```
#define CUSTOM_FVF  
D3DFVF_XYZ|D3DFVF_NORMAL|D3DFVF_DIFFUSE|D3DFVF_TEX1  
struct Vertex  
{  
    float x,y,z;    // D3DFVF_XYZ  
    float nx,ny,nz; // D3DFVF_NORMAL  
    DWORD dif;      // D3DFVF_DIFFUSE  
    float u,v;      // D3DFVF_TEX1  
}
```

ต่อมาเราจะต้องมาคำนวณ Normal Vector จากจุด Vertex ของ Primitive (Polygon)
โดยเราจะนำจุด Vertex ของ Primitive ในตำแหน่ง P0 มาลบกับ P1 และ P0 มาลบกับ P2
จากนั้นจึงนำผลลัพธ์มาคูณ (Cross-Product) กันอีกทีแล้วนำไป Normalize ต่อจึงจะได้ Normal
Vector มาหนึ่งจุด ตอนนี้จะได้แค่ Face Normal เอง

```

/** Normal_vector = Normalize((p0-p1)*(p0-p2)
vec3d ComputeNormalVector(vec3d p0,vec3d p1,vec3d p2)
{
    vec3d vNormal,v1,v2;
    v1 = Subtract(p0,p1);
    v2 = Subtract(p0,p2);
    vNormal = CrossProduct(v1,v2);
    return Normalize(vNormal);
}

```



รูปที่ 1 แสดง Face ที่เกิดจาก Vertex สามจุด

ต่อมาเราจะมาหา Vertex Normal วิธีการที่เราใช้ในการหา Vertex Normal ก็คือการนำผลรวมของ Face Normal ของ Polygon ที่มีจุด Vertex (ที่จะหา Vertex Normal) นั้นประกอบอยู่ทั้งหมดหารด้วยจำนวน Polygon ที่มีจุด Vertex นั้นประกอบอยู่ เมื่อนำมาเขียนเป็นโค้ดจะได้ดังนี้

```

void ComputeVertexNormals(vec3d* pVertices,unsigned int* pIndices,vec3d* pNormal,
                        int VertexCount,int PrimitiveCount)
{
    vec3d* pTempNormal,vSum;
    float share = 0.0f;
    int i,j;
    pTempNormal = new vec3d[PrimitiveCount];
    pNormal = new vec3d[VertexCount];
    for(i=0;i<PrimitiveCount;i++)
    {
        pTempNormal[i] = ComputeNormalVector(pVertices[pIndices[i*3]]
                                            ,pVertices[pIndices[i*3+1]]
                                            ,pVertices[pIndices[i*3+2]]);
    }
    for(i=0;i<VertexCount;i++)
    {
        vec3d v = pVertices[i];
        vSum = Set(0.0f,0.0f,0.0f);
        share = 0.0f;
        for(j=0;j<PrimitiveCount;j++)
        {
            if((pVertices[pIndices[j*3]] == v|| pVertices[pIndices[j*3+1]] == v||
                pVertices[pIndices[j*3+2]] == v)
            {
                vSum = Add(vSum,pTempNormal[j]);
                share++;
            }
        }
        pNormal[i].x = vSum.x/share;
        pNormal[i].y = vSum.y/share;
        pNormal[i].z = vSum.z/share;
    }
    delete[] pTempNormal;
}

```

เมื่อเราได้ Vertex Normal มาแล้วเราก็จะนำ Vertex Normal ใส่งไปในพิกัดจุด Vertex ของ Primitive ที่เราได้กำหนดโครงสร้างข้อมูลไว้ในก่อนหน้านี้ จากนั้นจึงทำการสร้าง Vertex Buffer และบรรจุ Vertex ที่มี Vertex Normal แล้วลงไปใน Buffer ต่อไป

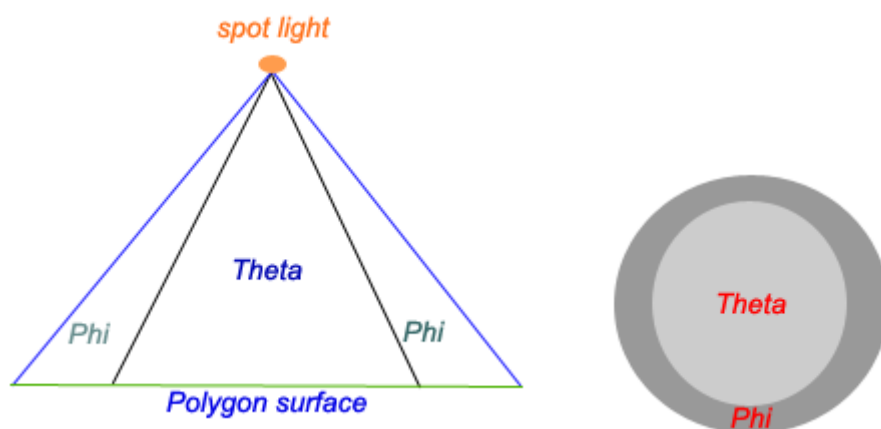
แหล่งแสงใน DirectX SDK

แหล่งแสงใน DirectX SDK (Direct3D) มีหลายชนิดทั้ง ambient, directional, point และ spot light แหล่งแสงแต่ละประเภคนั้นมีรูปแบบและการใช้งานแตกต่างกัน เริ่มจาก ambient light ซึ่งเป็นแหล่งแสงที่เป็น background light ในฉาก 3 มิติ นั้นหมายความว่าวัตถุ 3 มิติทุกๆ วัตถุจะได้รับแสงนี้ แหล่งแสงประเภทนี้จะไม่มีตำแหน่งใน world space เหมือนแหล่งแสงในประเภทอื่นๆ และเป็นแหล่งแสงที่ใช้พลังในการประมวลผลน้อยที่สุดการกำหนด ambient light ทำได้โดยใช้เมธอด SetRenderState ดังตัวอย่างเมื่อ r g และ b เป็นค่าแดง สีเขียว น้ำเงิน ตามลำดับซึ่งมีค่าอยู่ในช่วง 0- 255

```
gD3dDevice->SetRenderState( D3DRS_AMBIENT, D3DCOLOR_XRGB(r,g,b) );
```

point light เป็นแหล่งแสงที่เป็นจุด (ชื่อก็บอกอยู่แล้ว) ที่มีตำแหน่งใน world space ลักษณะแสงของ point light จะเป็นแสงที่เปล่งออกมารอบตัวในทุกๆ ด้าน แหล่งแสงประเภทนี้จะ เป็นแหล่งแสงที่ใช้พลังในการประมวลผลค่อนข้างมาก แต่ก็ให้ผลลัพธ์ที่น่าพอใจ

spot light เป็นแหล่งแสงที่เปล่งออกมาในลักษณะเป็นรูปกรวย (ตามรูปครับ) โดยรูป กรวยนั้นจะมีสองส่วนคือ กรวยชั้นใน (Inner Cone) มีความสว่างกว่ากรวยชั้นนอก (Outer Cone) โดยองศาความกว้างของกรวยชั้นในนั้นเราจะเรียกว่า Theta และ องศาความกว้าง ของกรวย ชั้นนอกซึ่งมีความกว้างมากกว่านั้นเราจะเรียกว่า Phi แหล่งแสงประเภทนี้จะ เป็นแหล่งแสงที่ใช้ พลังในการประมวลผลค่อนข้างมากเช่นกัน



รูปที่ 2 แสดงการฉายแสงของ Spot light

Directional light เป็นแหล่งแสงที่มีลักษณะคล้ายๆ กับแสงจากดวงอาทิตย์ (ที่เปล่งออกมา) จะมาในลักษณะทางเดียวโดยตรง ซึ่งจะตรงกันข้ามกับ point light แสงชนิดนี้ไม่มีจุดสิ้นสุด และเป็นแหล่งแสงที่ใช้พลังในการประมวลผลน้อยกว่า point light และ spot light

การกำหนด Material ให้แก่วัตถุ 3 มิติ

ก่อนที่จะสร้างแหล่งแสงให้แกโลก 3 มิติจะต้องมากำหนด Materials ให้แก่วัตถุ 3 มิติก่อน ถ้าหากใครเคยใช้โปรแกรม 3D Studio Max หรือโปรแกรม 3 มิติอื่นๆ ก็น่าจะเข้าใจหลักการของการกำหนด Materials ดี สำหรับ DirectX SDK การกำหนด Materials เป็นการกำหนดรูปแบบและคุณสมบัติของวัตถุ 3 มิติที่จะเกิดขึ้นเมื่อได้รับแสงมาตกกระทบบนพื้นผิว เช่น ถ้าหากวัตถุเป็นเหล็กหรือพลาสติกก็ย่อมจะมีความมันวาวเมื่อได้รับแสงซึ่งจะต้องตรงกันข้ามกับก้อนหินหรือไม้ที่จะไม่มีความมันวาวเมื่อได้รับแสง เป็นต้น ประโยชน์ของการกำหนด Materials ให้แก่วัตถุ 3 มิติคือจะทำให้รูปแบบของวัตถุ 3 มิติมีความหลากหลายและดูสมจริงมากยิ่งขึ้น การกำหนด Materials ให้แก่วัตถุ 3 มิตินั้นสามารถทำได้โดยใช้เมธอดของ Direct3DDevice

```
HRESULT SetMaterial(CONST D3DMATERIAL9 * pMaterial);
```

pMaterial เป็นพอยเตอร์ที่ชี้ไปยังโครงสร้างข้อมูล D3DMATERIAL9 ที่เราจะต้อง

กำหนดเข้าไปโดยโครงสร้างภายในของ D3DMATERIAL9 เป็นดังนี้

```
struct{  
    D3DCOLORVALUE Diffuse;  
    D3DCOLORVALUE Ambient;  
    D3DCOLORVALUE Specular;  
    D3DCOLORVALUE Emissive;  
    float Power;  
}
```

สำหรับโครงสร้างภายในของ D3DCOLORVALUE เป็นดังนี้

```
struct{  
    float r;  
    float g;  
    float b;  
    float a;  
}
```

โดย Diffuse เป็นสีหลักที่จะต้องกำหนดให้กับ Materials เนื่องจากวัตถุ 3 มิติที่ได้จะมีสีตามสีที่กำหนดใน Diffuse ซึ่งค่าสี Diffuse ใน D3DMATERIAL9 จะมีความสัมพันธ์กับ Diffuse ใน D3DLight9 ต่อมาคือ Ambient เป็นสีที่มีความสัมพันธ์กับ ambient light ส่วน Specula เป็นสีที่สัมพันธ์กับ specula highlight ถ้าหากใครเคยเล่น DOOM 3 แล้วลองสังเกตผนังโลหะของสถานีวิภาคดูจะเห็นว่าจะมีความสว่างของพื้นผิวโลหะเมื่อเรานำไฟฉายไปส่องใกล้ๆ จะเป็น specula highlight และสีสุดท้ายคือ Emissive เป็นค่าสีที่วัตถุ 3 มิติเปล่งออกมาเมื่อได้รับแสง ซึ่งยังมีค่ามากการเปล่งแสงกลับออกมาก็จะยิ่งมาก วัตถุก็จะมีแวววาวมากขึ้น และสุดท้ายคือ Power เป็นปริมาณค่า specula ที่มีบนวัตถุ 3 มิติ

สำหรับค่า Diffuse ambient specula และ Emissive จะอยู่ในช่วง 0.0 - 1.0 การกำหนด Materials ให้กับวัตถุ 3 มิตินั้นคุณสามารถจะกำหนดให้มีความแตกต่างกันได้วิธีการกำหนดก็ง่ายๆ

```
gD3dDevice->SetMaterial( &m_material แรก);
```

Draw วัตถุ 3 มิติแรก

```
gD3dDevice->SetMaterial( &m_material ที่ 2);
```

Draw วัตถุ 3 มิติที่ 2

.

.

```
gD3dDevice->SetMaterial( &m_material ที่ n);
```

Draw วัตถุ 3 มิติที่ n

การสร้างแหล่งแสง

เมื่อเรากำหนด Vertex Normal และ Materials ให้กับวัตถุ 3 มิติแล้วเราก็พร้อมที่จะสร้างแหล่งแสงกันแล้ว โดยในการสร้างแหล่งแสงนั้นเราจะใช้เมธอด SetLight ในการกำหนดรูปแบบและคุณสมบัติของแสงนั้นๆ จากนั้นจึงบรรจุแหล่งแสงนั้นลงไปในโลก 3 มิติโดยใช้เมธอด LightEnable และอย่าลืมสั่งให้ DirectX ทำการคำนวณค่าแสงตกกระทบบนวัตถุ 3 มิติด้วยเมธอด SetRenderState โดยกำหนด State ให้เป็น D3DRS_LIGHTING และ value ให้เป็น TRUE

```
gD3dDevice->SetRenderState( D3DRS_LIGHTING, TRUE );  
  
HRESULT SetLight(  
    DWORD Index,  
    CONST D3DLight9 * pLight  
);  
  
Index    เป็นหมายเลขของแหล่งแสงที่จะใช้ ค่าจะเริ่มจาก 0 ถึงจำนวนแหล่งแสงที่มากที่สุดที่ Device จะสามารถสร้างได้  
  
pLight   เป็นพอยเตอร์ที่ชี้ไปยังโครงสร้างข้อมูล D3DLight9 ที่เราจะต้องกำหนดเข้าไปโดยโครงสร้างภายในของ D3DLight9 เป็นดังนี้
```

```
struct {  
    D3DLIGHTTYPE Type;  
    D3DCOLORVALUE Diffuse;  
    D3DCOLORVALUE Specula;  
    D3DCOLORVALUE Ambient;  
    D3DVECTOR Position;  
    D3DVECTOR Direction;  
    float Range;  
    float Falloff;  
    float Attenuation0;  
    float Attenuation1;  
    float Attenuation2;  
    float Theta;  
    float Phi;  
}
```

Type เป็นชนิดของแสงที่เราจะใช้

point light ให้กำหนดเป็น D3DLIGHT_POINT

spot light ให้กำหนดเป็น D3DLIGHT_SPOT

directional light ให้กำหนดเป็น D3DLIGHT_DIRECTIONAL

Diffuse Specula และ Ambient เป็นปริมาณค่าสีดังกล่าวที่เปล่งออกมาจากแหล่งแสงนี้

Position คือตำแหน่งของแสงใน world space หากกำหนดชนิดของแสงเป็น directional light จะไม่จำเป็นต้องกำหนด

Direction คือตำแหน่งใน world space ที่แสงจะส่องไปหากกำหนดชนิดของแสงเป็น point light จะไม่จำเป็นต้องกำหนด

Range คือระยะที่แสงส่องไปถึง

Falloff เป็นปริมาณการส่องสว่างระหว่างกรวยชั้นนอกกับกรวยชั้นในของ spot light

Theta เป็นองศาความกว้างของ กรวยชั้นในของ spot light มีค่าเป็น radians

phi เป็นองศาความกว้างของ กรวยชั้นนอกของ spot light มีค่าเป็น radians

HRESULT LightEnable(

DWORD LightIndex,

BOOL bEnable

);

LightIndex เป็นหมายเลขของแหล่งแสงที่จะใช้

bEnable กำหนดให้เป็น TRUE หากต้องการใช้งานและ FALSE ถ้าเลิกใช้งาน

ตัวอย่างข้างล่างนี้เป็นการกำหนดแสงแบบ spot light, point light และ directional light ตามลำดับ

```
//*****spot light *****
```

```
D3DLIGHT9 light;
```

```
ZeroMemory( &light, sizeof(D3DLIGHT9) );
```

```
light.Type = D3DLIGHT_SPOT;
```

```
light.Diffuse.r = 0.0f;
```

```
light.Diffuse.g = 0.0f;
```

```
light.Diffuse.b = 0.5f;
```

```
light.Diffuses.a = 1.0f;
```

```
light.Position = position
```

```
light.Direction = direction
```

```
light.Range=1.0f;
```

```
light.Theta=0.5f;
```

```
light.Phi=1.0f  
light.Falloff=1.0f;  
light.Attenuation0 = 1.0f;  
gD3dDevice->SetLight( 0, &light );  
gD3dDevice->LightEnable( 0, TRUE );
```

```
//*****point light *****  
  
D3DLIGHT9 light;  
ZeroMemory( &light, sizeof(D3DLIGHT9) );  
light.Type = D3DLIGHT_POINT;  
light.Diffuse.r = 0.0f;  
light.Diffuse.g = 0.0f;  
light.Diffuse.b = 0.5f;  
light.Diffuses.a = 1.0f;  
light.Position = position  
light.Attenuation0 = 0.1f;  
light.Range = 200.0f  
gD3dDevice->SetLight( 1, &light );  
gD3dDevice->LightEnable( 1, TRUE );
```

```
//*****directional light *****  
  
ZeroMemory( &light, sizeof(D3DLIGHT9) );  
light.Type = D3DLIGHT_DIRECTIONAL;  
light.Diffuse.r = 0.0f;  
light.Diffuse.g = 0.0f;  
light.Diffuse.b = 0.5f;  
light.Diffuse.a = 1.0f  
light.Range = 1000.0f;  
D3DXVECTOR3 vecDir;  
D3DXVec3Normalize(&vecDir,& direction);  
light.Direction = vecDir;  
gD3dDevice->SetLight( 2, &light );  
gD3dDevice->LightEnable( 2, TRUE );
```

จำนวนแหล่งแสงในฉาก 3 มิติ

จำนวนแหล่งแสงที่ใช้ในฉากมีจำนวนจำกัดไม่สามารถกำหนดจำนวนแหล่งแสงได้ตามอำเภอใจ เราสามารถตรวจสอบจำนวนแหล่งแสงสูงสุดที่ใช้ได้ของ Device (การ์ดจอที่ใช้รันโปรแกรม) นั้นๆ ได้โดยใช้เมธอด `GetDeviceCaps` ซึ่งจำนวนแหล่งแสงมากที่สุดที่สามารถกำหนดได้จะอยู่ในตัวแปร `MaxActiveLights` ของโครงสร้างข้อมูล `D3DCAPS9` ครับ

Dynamic Light

ถ้าหากสังเกตการสร้างแหล่งแสงใน DirectX นั้นจะเห็นว่าจะต้องกำหนดสี ชนิด และ ตำแหน่ง รวมทั้งพารามิเตอร์ปลีกย่อยต่างๆ (ตามแต่ชนิดของแหล่งแสง) ของแสงก่อนถึงจะสร้างแสงได้ ทำให้แหล่งแสงที่สร้างขึ้นเป็นแสงที่ไม่มีการเปลี่ยนแปลง แต่ถ้าหากสังเกตเกมบางเกมจะเห็นได้ว่าแหล่งแสงของเขาสามารถเปลี่ยนทั้งตำแหน่งและสีได้ในทุกๆ เวลา ซึ่งแสงแบบนี้เราจะเรียกว่า Dynamic Light

2.3 Shading

คือการทำรายละเอียดของตัวพื้นผิว เช่น ความมันวาว การสะท้อนของพื้นผิว หรือความโปร่งแสง ทึบแสงของวัตถุ เป็นการให้สีเป็นลำดับขั้น

- Flat Shading Lighting เป็นการรายละเอียดพื้นผิวที่มีสีเหมือนกันทั่วทั้ง polygon
- Vertex Shading หรือ Gouraud Shading เป็นการให้สีแก่ vertex แต่ละจุดตามสีที่ได้กำหนดเอาไว้แล้ว

2.4 Texture

คือลวดลายของพื้นผิว โดยที่จะเป็น Bitmap ที่เป็น Pattern หรือ Image มักจะเก็บในรูปแบบไฟล์ BMP, PCX หรือ GIF เพื่อเป็นการใส่รายละเอียดให้แก่พื้นผิวของวัตถุ ทำให้วัตถุมีความสมจริง

- Texture Coordinate ใช้กำหนดการเชื่อมต่อกันระหว่าง Vertices ของ Face กับ Pixel ของ Bitmap โดย Texture Coordinate นี้ใช้แทน 2 มิติของ Coordinate System
- Texture Mapping คือ การวาดรูปลงบนพื้นผิวของ Face หรือ Polygon และในการทำ Texture Mapping นี้ต้องคำนึงถึงการคำนวณค่าต่างๆด้วย จึงต้องมีการกำหนดค่าของ Vertices ด้วย จากการที่ Texture Coordinate กำหนด Pixel ของ Texture ที่จะวาดลงในส่วนของ Face แล้วก็จะมีการ Wrapping เพื่อ Generate Texture Coordinate สำหรับ Object นั้นซึ่งการ Wrapping นั้นมี 5 ชนิดคือ

1. Flat Warp จะทำการวาดลงบน Face โดยตรง
2. Cubical Warp จะทำการ Warp Texture ใน Cube รอบๆ Object เหมือนกับการแปะลวดลายบนวัตถุที่มีลักษณะเป็นกล่อง
3. Cylindrical Warp จะทำการ Warp Texture ใน Cyline รอบๆ Object คือจะนำลวดลายในการ Map รอบโมเดลในลักษณะการห่อเหมือนทรงกระบอก ซึ่งเหมาะกับวัตถุที่มีลักษณะเป็นทรงกระบอกหรือเป็นแท่งๆ
4. Spherical Warp จะทำการ Warp Texture ใน Sphere Object ซึ่งจะนำภาพมาแปะในลักษณะห่อรอบทรงกลม เหมาะกับการแปะลวดลายบนวัตถุทรงกลม
5. Shrink Wrap คล้ายกับการ Map แบบ Spherical แต่โปรแกรมจะรวมจุดปลายของลวดลายที่ นำมา Map เข้าหากันเป็นจุดเดียว

2.5 Camera

Camera คือ มุมกล้องที่ทำให้เราสามารถมองเห็นวัตถุจากทิศทางต่างๆ เปรียบเสมือนว่าเรากำลังนำกล้องไปตั้งไว้บริเวณพื้นที่ที่มีวัตถุตั้งกล่าว ดังนั้นการเปลี่ยนมุมกล้องก็จะทำให้ภาพที่ปรากฏบนจอมีลักษณะแตกต่างกันไป โดยมุมกล้องจะถูกกำหนดโดย Viewport ซึ่งมีค่าต่างๆอยู่ 3 ค่าด้วยกันดังนี้

1. Eyepoint เปรียบเสมือนเป็นจุดที่กล้องตั้งอยู่
2. Lookat เป็นจุดที่กล้องมองไป

Upvector คือ Vector ที่ชี้บอกทิศด้านบนของกล้อง

2.6 DirectX

Direct X เป็น API ของ Microsoft พัฒนาเพื่อใช้จัดเตรียม Interface สำหรับควบคุมฮาร์ดแวร์มัลติมีเดียบนระบบ Microsoft Windows ได้อย่างมีประสิทธิภาพ และเป็นเครื่องมือให้โปรแกรมเมอร์ทำงานกับคำสั่งและโครงสร้างข้อมูลในระดับใกล้ชิดฮาร์ดแวร์ โดยไม่ต้องสร้างโค้ดติดต่อในระดับล่างซึ่งวิธีติดต่อจะแตกต่างกันไปตามประเภทของอุปกรณ์ การเขียนโค้ดที่เป็นอิสระจากอุปกรณ์ในลักษณะนี้ ช่วยให้โปรแกรมเมอร์สามารถสร้างซอฟต์แวร์เพื่อทำงานดังกล่าวได้เป็นอย่างดี แม้ผู้ใช้จะปรับเปลี่ยนอุปกรณ์ตัวใหม่และเพิ่มการ์ดเร่งความเร็วแบบสามมิติ, เสียง, อุปกรณ์อินพุตและอื่นๆก็ตาม

Microsoft direct X SDK (Software Development Kit) นั้นเป็นเครื่องมือที่ใช้ในการพัฒนาซอฟต์แวร์ทางมัลติมีเดียของบริษัท Microsoft ซึ่งรวมถึงสามารถพัฒนาเกมคอมพิวเตอร์ได้อีกด้วยโดยเริ่มต้นเราจะเห็นเฉพาะเฮดเดอร์ไฟล์และไลบรารีไฟล์ของ Direct Draw เท่านั้น หลังจากนั้นต่อมา งานด้าน 3 มิติ เริ่มเข้ามามีบทบาทมากขึ้น ทางบริษัท Microsoft จึงได้ทำการติดต่อกับทางบริษัทอังกฤษที่ชื่อว่า Reality Lab เพื่อขอซื้อ Rendermorphics มาจัดการในงานกราฟิก 3 มิติ ไว้ใน Microsoft DirectX SDK และพัฒนาเวอร์ชันใหม่ขึ้นเรื่อยๆ อันได้แก่ Microsoft DirectX SDK เวอร์ชัน 3, เวอร์ชัน 5 , เวอร์ชัน 6 , เวอร์ชัน 7, เวอร์ชัน 8 และปัจจุบันนี้อยู่ใน เวอร์ชัน 9 ทำให้มี Interface ใหม่ ๆ ขึ้นมา รวมถึงการเพิ่ม Method บางอย่างเข้าไปอีก เพื่อตอบสนองการใช้งานให้มากที่สุดนั่นเอง

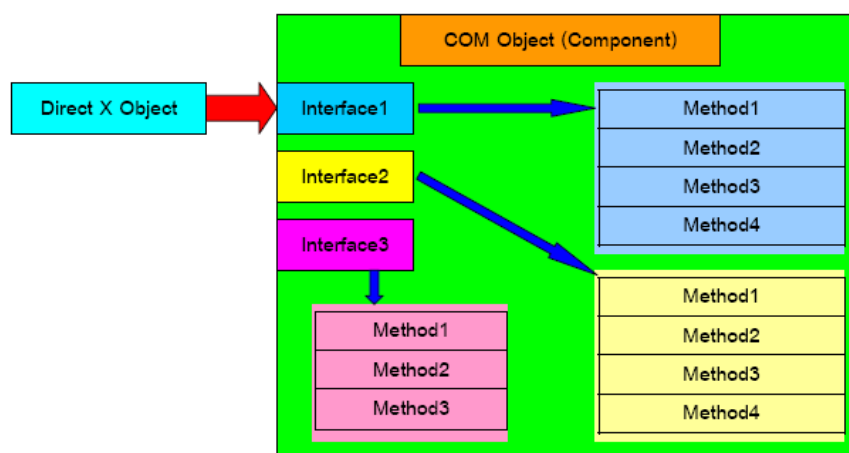
Component Object Model (COM)

COM เป็นวิธีที่ Microsoft คิดขึ้นมาเพื่อให้ Object ต่างๆ ที่อาจจะไม่มีความเกี่ยวข้องกันสามารถติดต่อสื่อสารแลกเปลี่ยนข้อมูลกันได้ โดยไม่ขึ้นกับภาษาที่ใช้เขียน, ไม่ขึ้นกับ OS, ไม่ขึ้นกับระบบของเครื่องคอมพิวเตอร์ Component คือการรวมเอา Object ย่อยๆ ต่างๆ ที่มีหน้าที่การทำงานในแบบเดียวกัน มารวมกันเป็น Component โดยเรียก Object ย่อยๆ เหล่านั้นใหม่ว่า Interface

Interface ของ COM คล้ายคลึงกับ Abstract Class ของภาษา C++ คือมีแต่ส่วนที่เป็นสัญลักษณ์ ความหมาย ข้อกำหนดความสามารถที่จะทำ แต่จะไม่มีโค้ดที่ใช้ทำงานแต่อย่างใด ทั้ง Interface ของ COM และฟังก์ชันแบบวิซวลของ C++ ทั้งสองใช้ไวยักษ์ตัวหนึ่งที่มีชื่อว่า vtable สำหรับพักแอดเดรสของฟังก์ชันหรือ Method สำหรับนำไปใช้ในการทำ Interface การที่ object จะสามารถใช้ Method ที่เป็นของ Interface ของ COM ได้ จะต้องเรียกใช้ Method QueryInterface เพื่อตรวจสอบว่า Interface ที่ต้องการใช้มีอยู่ใน Object หรือไม่ แล้วรับพอยเตอร์ของ Interface ดังกล่าวกลับมาใช้งานที่ 3591 .พอยเตอร์ที่ได้มาจากการติดต่อ vtable ด้วย Method QueryInterface5 ใน DirectX เวอร์ชัน 9.0 จะประกอบด้วย Component 8 ตัว คือ DirectX Graphics, DirectInput, DirectPlay, DirectSound, DirectMusic, DirectShow, DirectSetup, DirectX Media Objects ซึ่งจะไม่มีส่วน Component ที่ชื่อ DirectDraw, Direct3D โดยจะรวมเอา DirectDraw และ Direct3D ไปเป็น Interface อันเดียวคือ Direct3D Interface ซึ่งอยู่ใน DirectX Graphics Component

DirectX Object, COM Object, Interface และ Method

Interface แต่ละอันก็จะมี Method หรือฟังก์ชันการทำงานต่างๆ การจะเรียกใช้ Method นั้นต้องสร้าง DirectX Object ที่ชี้ไปยัง Interface ที่เราต้องการ โดยการสร้าง DirectX Object นี้สามารถทำได้โดยการเรียกใช้ Function หรือ Method ที่ DirectX เตรียมไว้ให้



รูปที่ 3 แสดงการเรียกใช้งานของ DirectX

จากรูป เปรียบเสมือนกล่องที่มีรูหรือช่องให้เสียบหลายๆ รู แต่ละรูก็เปรียบเสมือน Interface โดยแต่ละ Interface ก็มี Method ซึ่งก็คือหน้าที่การทำงานแบบต่างๆ เมื่อเราเปลี่ยนให้ DirectX Object เข้าไปที่ Interface ใด เราก็จะสามารถเรียกใช้งาน Method ใน Interface นั้นได้ ถ้าจะใช้ Method อื่นๆ ที่ไม่ได้อยู่ใน Interface ที่ DirectX Object ชี้อยู่ ก็ให้หา DirectX Object มาเพิ่มเพื่อจะเรียกใช้ Interface ได้และถ้าต้องการยกเลิกการใช้ Interface ใด เราก็ต้องถอดตัวที่ออก ข้อกำหนดหนึ่งของระบบ COM ก็คือ ถ้ามีการเผยแพร่ หรือแจกจ่าย COM ให้คนอื่นนำไปใช้แล้ว(เช่น ในกรณีที่ Microsoft ออก DirectX เวอร์ชันล่าสุดให้พวกเราได้ดาวน์โหลดมาใช้กัน) ถ้าหากในภายหลังต้องการจะแก้ไข Interface บางอันให้ทำงานได้ดียิ่งขึ้น จะต้องห้ามเข้าไปแก้ไขใน Interface เดิมให้ใช้วิธีสร้าง Interface ใหม่เพิ่มขึ้นมาแทน (โดยอาจจะสืบทอดมาจาก Interface เดิม) เพราะถ้าไปแก้ไขอาจจะทำให้โปรแกรมเก่าๆ ที่เขียนด้วย DirectX SDK เวอร์ชันเก่า ซึ่งมีการเรียกใช้ Interface อันเดิมใช้งานไม่ได้

นอกจากนี้ยังต้องตั้งชื่อ Interface ที่สร้างใหม่ไม่ให้ซ้ำกับของเดิมด้วย จุดนี้เองเราจึงเห็นชื่อ Interface ใน DirectX ใช้วิธีใส่ตัวเลขเวอร์ชันกำกับไว้ข้างท้าย เช่น IDirect3D8, IDirect3D9 เป็นต้น DirectX จะใช้ตัวอักษร I นำหน้าชื่อ Interface) ดังนั้น ใน Component ของ DirectX เวอร์ชันใหม่ๆ ก็จะมี Interface ของเวอร์ชันเก่าๆ รวมอยู่ด้วย จึงทำให้เกมที่สร้างด้วย DirectX เวอร์ชันก่อนหน้านี้สามารถรันบน DirectX เวอร์ชันใหม่ๆ ได้ แต่ผลลัพธ์ของเกม เช่น Effect ต่างๆ หรือแม้กระทั่งความเร็วของเกม ก็ยังคงใกล้เคียงกับที่เคยรันด้วย DirectX เวอร์ชันเดิม เพราะ Interface มันเก่าก็ยังไม่เปลี่ยนแปลง Method ต่างๆ ก็ไม่เปลี่ยนแปลง

API ของ DirectX 9.0 SDK

DirectX 9.0 SDK ถูกพัฒนาให้มีความสามารถเพิ่มขึ้นจาก เวอร์ชันก่อนๆ โดยสามารถแบ่งออกเป็นส่วนประกอบหลักๆได้ดังนี้

1 DirectX Graphics เป็นการรวมกันระหว่าง DirectDraw และ Direct 3D โดยมีจุดเด่นตรงที่การประมวลผล กราฟิกแบบขนาน และสามารถจัดการกับกราฟิกให้มีความสมจริงกว่าเวอร์ชันที่ผ่านมาทำให้ผู้พัฒนาเกม สามารถพัฒนาเกม 3มิติ ได้ง่ายขึ้น แต่สำหรับการพัฒนาเกม 2 มิติ กลับทำได้ยุ่งยากขึ้น

2 DirectInput เป็นอีกองค์ประกอบหนึ่งของ Direct X ที่ช่วยจัดการในส่วนของการจัดการอุปกรณ์แป้นพิมพ์ (Keyboard), เมาส์ (Mouse) และ จอยสติ๊ก (Joy Stick) ซึ่งเป็นส่วนที่ช่วยให้โปรแกรมเมอร์ สามารถที่จะติดต่อกับอุปกรณ์ดังที่ได้กล่าวมาได้โดยง่ายโดยในการเขียนโปรแกรมติดต่อกับอุปกรณ์ Keyboard หรือ Mouse ก็จะมีขั้นตอนของการเขียนโปรแกรมหาดังนี้ คือ

- สร้าง Keyboard หรือ Mouse Device
- Set Cooperation level ให้กับ Keyboard เพื่อกำหนดรูปแบบการใช้งานอุปกรณ์
- Set รูปแบบข้อมูลให้กับ Keyboard Device หรือ Mouse Device
- Acquire Keyboard Device หรือ Mouse Device

ในเวอร์ชันนี้ได้มีการเพิ่มประสิทธิภาพใหม่เข้าไป คือสามารถใช้ Action Mapping ได้ซึ่งใช้กำหนดค่าต่างๆให้กับอุปกรณ์ควบคุมและยังสามารถจัดการเชื่อมต่อระหว่าง InputAction กับอุปกรณ์ควบคุม(Input Device)ได้และการเปลี่ยนแปลงค่าทำได้ง่ายขึ้น โดยจะขึ้นอยู่กับชนิดของอุปกรณ์ควบคุม(Input Device) นั้นๆ นอกจากนี้ยังได้เพิ่มการสนับสนุนอุปกรณ์ต่างๆได้มากขึ้น

3 DirectPlay ถูกเพิ่มประสิทธิภาพในการสนับสนุนการทำงานในรูปแบบเน็ตเวิร์กหรือระบบเครือข่ายให้มีความหลากหลายในการใช้งาน อีกทั้งยังเพิ่มการป้องกันความผิดพลาดที่อาจเกิดขึ้นได้กับระบบนอกจากนี้DirectPlayยังเพิ่มการสนับสนุนการสื่อสารด้วยเสียงระหว่างผู้เล่น (Voice Communication)

4 DirectSound เป็นชุด API ที่มีเครื่องมือในการใช้จัดการกับระบบเสียงแบบสเตอริโอและแบบ 3 มิติ ได้อย่างมีประสิทธิภาพประกอบด้วยความสามารถในการจัดการหน่วยความจำ การผสมเสียงของฮาร์ดแวร์ DirectSound ได้รับการออกแบบมาเพื่อดึงความสามารถของฮาร์ดแวร์ระบบ การรวมเสียงแบบ 3 มิติเข้าไปในเกมหรือการซิมูเลชัน ทำให้แอปพลิเคชันสามารถให้เสียงได้สมจริงสมจัง การได้ยินเสียงจากทางซ้าย ทางขวา หรือจากทางด้านบน โดยเฉพาะอย่างยิ่งเสียงที่มาจากรอบ ๆ ตัวจะช่วยสร้างสภาพแวดล้อมที่ตื่นตาตื่นใจมากขึ้น

5 DirectMusic เป็นชุด API ที่ทำงานกับข้อมูลประเภท Message-base Musical Data ซึ่งเป็นข้อมูลที่แปลงมาจาก Wave Sample ด้วยซินธิไซเซอร์ (Synthesizer) ทั้งแบบฮาร์ดแวร์หรือซอฟต์แวร์ โดยปกติถ้าเป็นซอฟต์แวร์จะใช้โปรแกรม Microsoft Software Synthesizer เพื่อสร้าง Wave Sample ให้กับ DirectSoundการใช้เสียงดนตรีที่ได้จากซินธิไซเซอร์จะเป็นไปตามมาตรฐาน DLS (Downloadable Sound) นอกจากนี้ DirectMusic ยังมีกลไกที่ใช้สร้างเพลงได้ตามที่เรา กำหนด

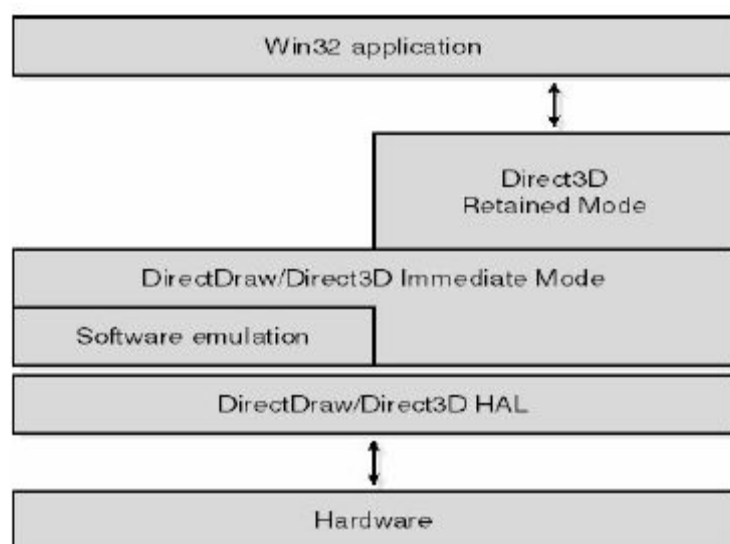
6 DirectShow ได้ถูกเพิ่มเข้ามาตั้งแต่ Version DirectX 8.0 นี้โดยในเวอร์ชันก่อนหน้านี้ DirectShow จะถูกแยกต่างหากจาก DirectX โดยจะทำหน้าที่ในการเล่นไฟล์มัลติมีเดีย ประเภท ภาพวีดิโอเป็นหลัก แต่เมื่อรวมเข้ากับ DirectX แล้ว ทำให้การเล่นไฟล์มัลติมีเดียประเภทภาพวีดิโอ สามารถควบคุมได้ง่ายและมีประสิทธิภาพมากขึ้น

7 DirectSetup API ชุดนี้จะทำหน้าที่เกี่ยวกับการติดตั้งคอมโพเนนต์ DirectX Runtime ให้กับระบบโดยอัตโนมัติถ้าหากระบบดังกล่าวยังไม่ได้รับการติดตั้ง แม้ในปัจจุบันมักจะมีการ ติดตั้งคอมโพเนนต์มากับระบบปฏิบัติการแล้วก็ตาม แต่สำหรับ DirectX แล้วไมโครซอฟท์ได้ อนุญาตให้ใช้โดยที่ไม่ต้องเสียค่าใช้จ่าย จึงไม่ต้องกังวลเรื่องของลิขสิทธิ์ โดยสามารถทำการติดตั้ง ที่ครั้งก็ได้

8 Direct Media แม้ว่าโปรแกรมเมอร์สามารถเขียนโปรแกรมติดต่อกับ ActiveX Control โดยตรงได้โดยใช้ฟังก์ชันของ COM Interface แต่มันก็ออกจะยุ่งยากไปบ้าง ดังนั้นเพื่อความสะดวก ในการใช้งาน โปรแกรมเมอร์สามารถเลือกใช้ DirectX Media SDK ซึ่งมีคลาสของ MFC ที่มีชื่อว่า CMediaPlayer ให้โปรแกรมเมอร์เลือกใช้แทนที่จะเรียก COM Interface โดยตรง ทำให้การใช้งาน Media Player Control ได้ง่ายมากขึ้น

การทำงานของ Direct3D

Direct3D มีขึ้นครั้งแรกใน DirectX 2.0 ประมาณช่วงปี ค.ศ. 1996 หลังจากนั้นภายในระยะเวลาอันสั้น Direct3D ได้กลายเป็นชุด API สำหรับการทำงานที่ได้รับความนิยมอย่างสูงในท้องตลาดที่เกมและซิมูเลชันส่วนใหญ่เลือกใช้ รวมทั้งได้รับการสนับสนุนจากผลิตภัณฑ์เร่งความเร็ว 3 มิติ ปัจจุบันฮาร์ดแวร์เร่งความเร็วกราฟิกพัฒนาไปจนสามารถแสดงโพลีก่อนได้หลายล้านโพลีก่อนต่อวินาที ฮาร์ดแวร์เร่งความเร็วที่นำมาเชื่อมการทำงานกับ Direct3D มีความเร็วมากขึ้น และช่วยลดต้นทุนของแอปพลิเคชัน 3 มิติ ทำให้ไม่ต้องทุ่มทุนสูงนักเพื่อแลกกับประสิทธิภาพในการทำงานที่สูงขึ้น เช่นเดียวกับ API ทั้งหมดของ DirectX ชุด API ของ Direct3D ไม่ได้ออกแบบมาให้เป็นเพียง API สำหรับการพัฒนาเกมเพื่อรองรับความสามารถที่จะมีขึ้นในอนาคตเท่านั้น แต่ยังได้รับการออกแบบมาให้มีความเข้ากันได้ทั้งหมดกับเวอร์ชันที่ผ่านมา ดังนั้นเกมที่พัฒนาด้วย DirectX เวอร์ชันเก่าก็จะสามารถรันได้ใน DirectX เวอร์ชันที่จะมีขึ้นในอนาคต



รูปที่ 4 แสดงการติดต่อจากระดับ Application ไปถึงระดับ hardware โดยผ่าน DirectX

Direct3D ในยุคแรกมี API อยู่สองโหมดคือ โหมด Immediate(IM) และโหมด Retained(RM) ในโหมด IM (การเรนเดอร์อ็อบเจกต์สามารถทำได้โดยฉับพลัน ได้ตามความต้องการของโปรแกรมเมอร์) เป็นโหมดที่ใช้งานยากแต่จะมีความยืดหยุ่นสูง เป็น API ในระดับล่างสำหรับใช้เขียนเกมที่ทำงานได้เร็วและมีประสิทธิภาพเท่าที่เครื่องจะเป็นไปได้บนระบบ ในโหมด RM (API จะเก็บขึ้นลงในฐานข้อมูลแล้วนำมาเรนเดอร์ทั้งหมดในคราวเดียวกัน) เป็นโหมดที่สร้างขึ้นมาเป็นเลเยอร์ที่อยู่บนสุดของโหมด IM โดยในโหมดนี้จะจัดเตรียมบริการต่าง ๆ เช่น การจัดการ Texture, การโหลด Object File, การจัดลำดับเฟรม และการทำ อ็อบเจกต์เคลื่อนไหว การศึกษาและการทำงานโหมด RM นั้นจะง่ายกว่าเมื่อเทียบกับการใช้งานในโหมด IM แต่ถ้าหากโปรแกรมเมอร์คนใดต้องการประสิทธิภาพและความยืดหยุ่นในการทำงานก็มักจะเลือกใช้งานในโหมด IM ดังนั้นการทำงานในโหมด RM จึงได้หยุดพัฒนาลงใน Direct X version 6.0 และหันไปมุ่งพัฒนาโหมด IM ให้มีความสามารถและใช้งานง่ายในเวอร์ชันต่อมา ด้วยเหตุนี้การทำงานในโหมด RM จึงไม่สนับสนุนเทคโนโลยีใหม่ เช่น Multitexturing, Bump Mapping, Hardware Transformation และ Lighting ดังนั้นความสามารถทั้งหมดของโปรแกรม 3 มิติ ควรเขียนขึ้นมาโดยใช้โหมด IM และในการทำงานร่วมกับฮาร์ดแวร์เร่งความเร็วกราฟิกของแอปพลิเคชัน Direct3D จะใช้การติดต่อกับฮาร์ดแวร์ผ่านทาง HAL (Hardware Abstraction Layer) แต่ถ้าในพีเจอนั้นไม่มีในฮาร์ดแวร์ Direct3D จะใช้ HEL (Hardware Emulation Layer) แทน

2.7 X File

เป็น File ประเภทหนึ่งโดยมีนามสกุลเป็น .x ซึ่งใช้ในการเก็บข้อมูลต่างๆ ของ Mesh ตั้งแต่จุด Vertices สีของ Vertices รายละเอียดการ Texture รวมไปถึง Frame Animation ด้วย

โดยใน X File นั้นจะมีการประกาศ Template ต่างๆ เพื่อใช้ในการระบุ Data type ของข้อมูล โดยมีรูปแบบดังนี้

```
template <template-name> {  
    <GUID>  
    <member 1>;  
    ...  
    <member n>;  
    [restrictions]  
}
```

ในการประกาศ Template ให้เริ่มด้วย Template ตามด้วยชื่อของ Template ต่อมาทำการกำหนดค่า GUID (Globally unique identifier) ซึ่งเป็นชุดของตัวเลขและตัวอักษร ใช้ในการระบุและแยกแยะ Template ในการ Coding ตามด้วยการประกาศ Member Data Type ต่างๆ ตามจำนวนที่ต้องการ

Type	Size
WORD	16 bits
DWORD	32 bits
FLOAT	IEEE float
DOUBLE	64 bits
CHAR	8 bits
UCHAR	8 bits

BYTE	8 bits
STRING	NULL terminated string

รูปที่ 5 แสดง Data type ที่ใช้ใน X File

ตัวอย่าง Template

```
template MY_GAME_INFO{  
    <AA1308FD-FF98-4f6e-9A55-CD083178672F>  
  
    STRING GameName;  
  
    STRING GameMaker;  
  
    DWORD Version;  
  
}
```

ตัวอย่าง Data Object ที่ใช้ในการเก็บข้อมูล

```
MY_GAME_INFO Game01  
{  
  
    "My Test Game";  
  
    "Alan Thorn";  
  
    1;  
  
}
```

นอกจาก Template ที่เราทำการสร้างเองได้แล้วใน DirectX ยังมี Standard Template ที่ถูกประกาศเอาไว้แล้วเตรียมเอาไว้ให้ใช้อยู่แล้วหลาย Template ด้วยกันเช่น

```
template Animation  
{  
    < 3D82AB4F-62DA-11cf-AB39-0020AF71E433 >  
  
    [...]  
  
}  
  
template AnimationKey  
{  
    < 10DD46A8-775B-11CF-8F52-0040333594A3 >  
  
    DWORD keyType;  
  
    DWORD nKeys;  
  
    array TimedFloatKeys keys[nKeys];  
  
}
```

เป็นต้น

2.8 Data structure

เนื่องจากในตัวเกมต้องมีการใช้และควบคุมจัดการกับข้อมูลต่างๆเป็นจำนวนมาก จึงจำเป็นต้องมีการจัดการกับข้อมูลที่ดีด้วย

ในตัวเกมใช้ Class ทั้งหมด 2 Class หลักๆในการจัดการข้อมูล ดังนี้

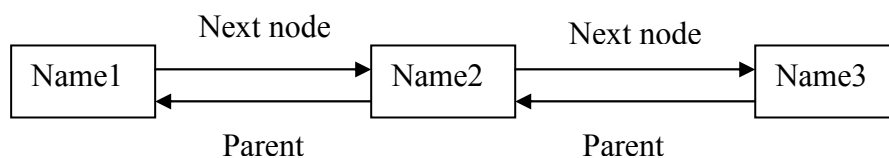
1. StoreMesh Class เป็น Class ที่ใช้ในการเก็บข้อมูลต่างๆ ของตัวละครทุกตัว โดยข้อมูลที่จัดเก็บมี ดังต่อไปนี้ พิกัดที่อยู่ของตัวละคร (x, y, z), ค่าการหันหน้าของตัวละคร (ry), ชื่อของตัวละคร, ค่าความเร็วในการโจมตี, ค่าDelayต่างๆ

2. Loadresource Class ใช้ในการเก็บข้อมูลของตัวโมเดล Frame Animation ต่างๆ โดยในช่วงแรกของโปรแกรมจะทำการโหลดข้อมูลทั้งหมดเก็บไว้ใน Class นี้ให้เรียบร้อยเพื่อให้สามารถนำไปใช้ในภายหลังได้โดยไม่ต้องโหลดซ้ำอีก

Link List

เนื่องจากเกมนี้เป็นเกม Online จำนวนผู้เล่นจึงมีไม่แน่นอน ทางผู้จัดทำจึงเลือก Link List มาใช้ในการเก็บข้อมูล ทำให้ไม่ต้องจองเนื้อที่มากเกินไปจนเกินไป

ในที่นี้เราได้นำ Link List มาใช้ใน StoreMesh Class โดยเมื่อมีผู้เล่นเข้ามาเพิ่มก็จะทำการ Add Node เพื่อเพิ่มจำนวนผู้เล่น ลักษณะการทำงานของ Link list จะเป็นการเก็บข้อมูลใน Class หนึ่ง แล้วจะมีตัวแปรใน Class นั้นเป็น Pointer ทำหน้าที่ชี้ไปยังตัวต่อไป ดังรูป



ตัวอย่าง Class ที่มีการใช้ Link List

Class test

```
{    public: LPCSTR Name;

        FLOAT x;

        FLOAT y;

        StoreMesh *nextmesh;

        StoreMesh *parent;

};
```

2.9 Network Load Balancing และ Clustering

Cluster คืออะไร

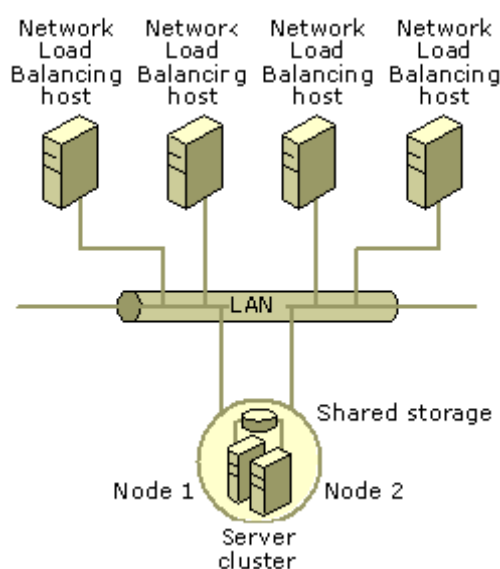
Cluster คือ การรวมกลุ่มเซิร์ฟเวอร์หลายๆ โหนดเข้าด้วยกันเสมือนว่าเป็นเซิร์ฟเวอร์ตัวเดียว ซึ่งในมุมมองของผู้ใช้อาจไม่มีโอกาสรับรู้ถึงการมีอยู่ของเซิร์ฟเวอร์ทั้งหลายภายในกลุ่มของ Cluster แต่จะเข้าใจว่าเป็นเซิร์ฟเวอร์ตัวเดียวกัน

Cluster ถูกสร้างขึ้นเพื่อรองรับระบบงานหรือแอปพลิเคชันเซิร์ฟเวอร์ต่างๆ ที่ใช้ในองค์กร โดยที่ทุกๆ เซิร์ฟเวอร์ในกลุ่มจะรู้จักแอปพลิเคชันเซิร์ฟเวอร์ที่รันอยู่บน Cluster

เทคโนโลยีของ Cluster นั้นแบ่งออกเป็น 2 แบบ ได้แก่

2.9.1 บริการแบบ Clustering (MSCS)

ลักษณะการทำงานของบริการแบบนี้ จะประกอบด้วยเซิร์ฟเวอร์หลักที่ทำงานอยู่ (Active) และเซิร์ฟเวอร์สำรองที่คอย Standby อยู่ โดยปกติ เซิร์ฟเวอร์หลักที่ Active จะเป็นโหนดที่ให้บริการแก่ผู้ใช้ เมื่อใดก็ตามที่เซิร์ฟเวอร์หลักไม่สามารถปฏิบัติงานได้ เช่น เกิดความผิดพลาดและดาวนลง ไป กระบวนการที่เรียกว่า FAILOVER จะเกิดขึ้นเพื่อกระตุ้นให้เซิร์ฟเวอร์ที่ Standby อยู่ขึ้นมารับช่วงทำงานแทนเพื่อให้บริการแก่ผู้ใช้ต่อไปอย่างต่อเนื่องไม่ขาดตอน ผู้ใช้เองจะรู้สึกหรือรับรู้ว่ามีเซิร์ฟเวอร์สำรองขึ้นมาทำงานแทน



รูปที่ 6 แสดงโครงสร้าง Clustering

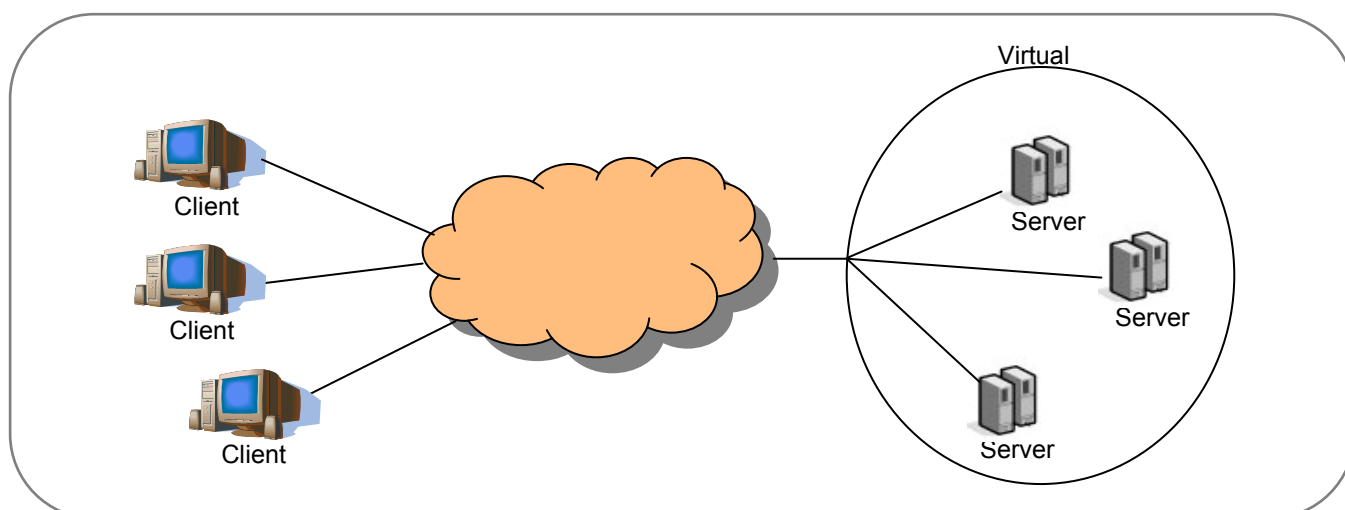
2.9.2 บริการแบบ Network Load Balancing (NLB)

Network Load Balancing คือ การเพิ่มขีดความสามารถในเรื่องการใช้งานและการขยายระบบ โดยเน้นในเรื่องการให้บริการพื้นฐานบนระบบเครือข่าย เช่น TCP/IP และการให้บริการต่างๆ เกี่ยวกับเว็บไซต์ ซึ่งสามารถแบ่งได้เป็น 2 แบบหลัก ดังนี้

แบบ Scalability หมายถึง การที่ระบบสามารถถูกเพิ่มขยายเพื่อรองรับปริมาณที่เพิ่มขึ้นของโหลดหรือ ทราฟฟิกที่วิ่งเข้ามาได้อย่างต่อเนื่อง หากโหลดมาก เราสามารถเพิ่มจำนวนของเซิร์ฟเวอร์เข้าไปเพื่อช่วยกันทำงานและแบ่งเบาภาระซึ่งกันและกันได้

แบบ Availability หมายถึง การที่ระบบสามารถให้บริการแก่ผู้ใช้ได้อย่างต่อเนื่อง ไม่หยุดชะงักในกรณีที่บางเซิร์ฟเวอร์ทำงานผิดพลาดไป เซิร์ฟเวอร์อื่นก็สามารถรับหน้าที่ทำงานแทนต่อได้

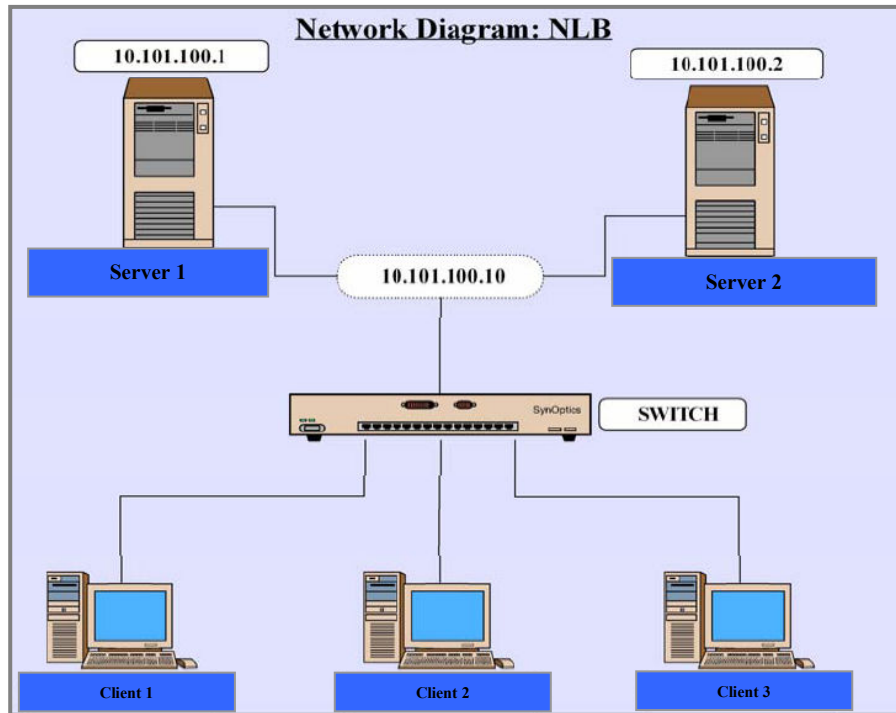
บริการแบบ NLB นี้เหมาะกับการประยุกต์ใช้ “เซิร์ฟเวอร์ฟาร์ม” หรือกลุ่มของเซิร์ฟเวอร์ที่รองรับระบบงานเฉพาะด้าน อย่างเช่น การทำเซิร์ฟเวอร์ฟาร์มให้กับเกมเซิร์ฟเวอร์



รูปที่ 7 แสดงหลักการทำงานของ Network Load Balancing

จากรูป

1. เมื่อเซิร์ฟเวอร์ได้รับคำร้องขอจากเครื่อง Client
2. มีการส่งกระจายคำร้องขอไปยังเซิร์ฟเวอร์ที่อยู่ภายใน Cluster เดียวกัน
3. จะมี Server หนึ่งที่สามารถจัดการกับคำร้องขอของเครื่อง Client ซึ่งผู้ใช้สามารถใช้งานได้อย่างต่อเนื่องตามปกติ



รูปที่ 8 แสดงการทำงานของ Network Diagram NLB

จากรูปแสดงการทำงานโดยรวมจะเป็นไปในลักษณะว่ามีการสร้าง Virtual Server (เซิร์ฟเวอร์เสมือน) ในที่นี้คือ 10.101.100.10 ซึ่ง Client จะทำการติดต่อเข้ามาที่ IP ของ Virtual Server นี้และ Virtual Server จะทำหน้าที่ Balance Load ที่เข้ามาให้กับ Server ทั้งสองตัว

ซึ่งข้อดีของโครงสร้างลักษณะนี้ คือ ในกรณีที่เกิดความผิดพลาดขึ้นกับ Server เครื่องใดเครื่องหนึ่ง และ Server อื่นยังสามารถทำงานได้ ระบบจะยังคงทำงานต่อไปได้ โดย Client Side อาจไม่ทราบเลยว่ามี Server ใดเกิดความผิดพลาดไป